

Universidad Nacional
Autónoma de México

Facultad de Ingeniería



Lab. Computación Gráfica e Interacción Humano-Computadora

Manual técnico

Proyecto Final - Tema: Cultural

Altar de Muertos en Cementerio

Datos académicos

Integrantes	Echeverria Goicochea, Erick Isaac [424153520]
	Cruz Miranda, Yasser Vladimir [424039512]
Profesor	Espinoza Urzúa, Edén
Grupo	4
Semestre	2027-1
Repositorio	`github.com/VvyasservV/Ofrenda-dia-de-muertos-laboratorio-de-CG`

Fecha de entrega: 09/11/2026

Índice

1. Introducción **2**

2. Objetivos **2**
2.1. Objetivo general . . . 2
2.2. Objetivos específicos . . . 2

3. Alcance y limitantes **2**
3.1. Alcance del proyecto . . . 2
3.2. Limitantes del proyecto . . . 3

4. Diseño y arquitectura del software **3**
4.1. Arquitectura general . . . 3
4.2. Diagrama de flujo . . . 4
4.3. Estructura de archivos y carpetas . . . 5
4.4. Metodología de software aplicada . . . 6
4.5. Planificación y cronograma (diagrama de Gantt) . . . 7

5. Modelado y recursos gráficos **8**
5.1. Descripción de los objetos modelados . . . 8
5.2. Texturas y materiales . . . 11
5.3. Iluminación y ambientación . . . 11

6. Documentación del código fuente **13**
6.1. Inicialización de OpenGL . . . 13
6.2. Sistema de renderizado . . . 13
6.3. Cámara y perspectiva . . . 14
6.4. Manejo de entradas del usuario . . . 14
6.5. Funciones principales . . . 15
6.6. Generación del ejecutable . . . 16

7. Sistema de animaciones **17**
7.1. Animación sencilla 1: viento en el papel picado (tecla 1) . . . 17
7.2. Animación sencilla 2: banda tocando (tecla 2) . . . 17
7.3. Animación sencilla 3: reja de la entrada (tecla 3) . . . 17
7.4. Animación compleja 1: niño que saluda (tecla 4) . . . 18
7.5. Animación compleja 2: mariposas monarca (tecla 5) . . . 19

8. Pruebas y resultados **20**
8.1. Pruebas funcionales . . . 20
8.2. Validación de objetos y animaciones . . . 20
8.3. Evidencias del resultado final . . . 20

9. Conclusiones **21**

10. Referencias (APA 7) **21**

11. Repositorio del proyecto **23**

1 Introducción

El proyecto es una recreación de un panteón mexicano en la noche de Día de Muertos, hecha para el proyecto final del laboratorio (Espinoza Urzúa, 2026) y tomando como referencia una escena tradicional con tumbas, flores de cempasúchil, velas y una ofrenda. Lo desarrollamos con OpenGL 3.3 y el código base del laboratorio, y en él juntamos los temas que vimos en el curso: modelos 3D con texturas, varias fuentes de luz, animaciones por código y controles para que el usuario recorra el panteón y prenda o apague algunos de sus elementos.

2 Objetivos

2.1 Objetivo general

Recrear en 3D, con OpenGL 3.3 y el código base del laboratorio, un panteón mexicano en la noche de Día de Muertos con una ofrenda tradicional, a partir de imágenes de referencia, aplicando lo aprendido en el curso: carga de modelos, texturizado, iluminación, modelado jerárquico, animación e interacción con el usuario.

2.2 Objetivos específicos

- Modelar y texturizar al menos siete objetos de la imagen de referencia con la mayor fidelidad posible.
- Armar la escena a partir de un archivo de datos, para colocar más de 1,200 objetos sin escribir una línea de código por cada uno.
- Iluminar la escena de noche con el modelo de Phong (luna, velas y linterna) y agregar un brillo de borde (Fresnel), niebla y un skybox nocturno.
- Programar cinco animaciones por código que tengan sentido en el contexto (tres sencillas y dos complejas), basadas en las prácticas de Animación básica, Máquina de estados y KeyFrames.
- Permitir que el usuario recorra la escena con teclado y ratón y que prenda y apague las luces y las animaciones.
- Entregar un ejecutable que no dependa de la carpeta Debug, junto con el manual de usuario y este manual técnico.

3 Alcance y limitantes

3.1 Alcance del proyecto

El proyecto incluye:

- Un recorrido libre en primera persona por el panteón, con la cámara del código base (`Camera.h`).
- La escena completa: barda y reja de la entrada, capilla, terrazas con escaleras, 20 tumbas de distintos tipos (entre ellas cuatro mausoleos), cruces, un ángel, angelitos, árboles dentro y fuera de la barda, la ofrenda de varios niveles con todos sus elementos, coronas, tapetes de flores, papel picado, guirnaldas, caminos de cempasúchil, 273 velas, 48 personas y una banda de siete músicos. En total son 49 modelos distintos y 1,208 objetos colocados.

- Cinco interacciones con la iluminación: velas, linterna, intensidad de la luna, brillo de borde y niebla.
- Cinco animaciones por código activadas con las teclas 1 a 5.
- El ejecutable en `build/`, el manual de usuario, este manual técnico y un video.

Quedan fuera del alcance el sonido, las colisiones con los objetos (la cámara puede atravesarlos), las sombras proyectadas, las personas que caminan por su cuenta y una interfaz gráfica con menús.

3.2 Limitantes del proyecto

El proyecto tiene las siguientes limitantes técnicas:

- **Código base obligatorio.** El proyecto debe usar el código del profesor (`Shader`, `Camera`, `Mesh`, `Model` y SOIL2), que carga modelos con assimp y no lee esqueletos. Por eso todas las animaciones se hacen con transformaciones por código sobre modelos separados en partes, y no con huesos.
- **Versión de assimp.** El assimp incluido en el código base es antiguo, así que los modelos se exportan a OBJ, que lee sin problemas.
- **Solo Windows y 32 bits.** Las librerías del código base (GLEW, GLFW y assimp) vienen compiladas para Win32, así que el proyecto se compila en Release | Win32.
- **Rendimiento.** Con más de 1,200 objetos y 24 luces puntuales por píxel, una tarjeta integrada puede bajar de cuadros por segundo. Agregamos optimizaciones (sección 6.2), pero la niebla y la cantidad de velas siguen siendo lo más costoso.
- **Tiempo de carga.** Los modelos se cargan al iniciar, por lo que el programa tarda algunos segundos en mostrar la escena.
- **Tamaño de los modelos.** El laboratorio pide que cada objeto pese menos de 100 MB; el más pesado (los pétalos del camino) pesa unos 8 MB.

4 Diseño y arquitectura del software

4.1 Arquitectura general

El proyecto parte de la práctica Fuentes de luz (`fuentesDeLuz.cpp`). De ahí vienen la ventana, el ciclo principal, la cámara, el manejo del teclado y del ratón, y el shader de Phong con luz direccional, luces puntuales y linterna. Sobre esa base, todo el programa vive en un solo archivo (`main.cpp`), que coordina la carga de modelos, la escena, las luces, las animaciones y la interacción; los shaders están en archivos aparte. La escena se lee de archivos de datos (sección 4.3), así que cada modelo se carga una sola vez y se dibuja en todas sus posiciones. La tabla 1 resume las herramientas que usamos.

Herramienta	Uso
C++ y Visual Studio 2026	Lenguaje y entorno de desarrollo (Release, Win32, conjunto de herramientas v145, el mismo del código base)
OpenGL 3.3 (perfil core) y GLSL 330	Dibujo de la escena y shaders
GLFW	Ventana, teclado y ratón
GLEW	Funciones de OpenGL
GLM	Matrices y vectores (transformaciones)
assimp	Carga de los modelos OBJ (en `Model.h` del código base)
SOIL2	Carga de texturas PNG y de las caras del skybox
Blender 5.2	Modelado, texturizado y exportación a OBJ
Git, GitHub y GitHub Desktop	Control de versiones y avances diarios

Tabla 1: Herramientas y tecnologías.

4.2 Diagrama de flujo

La figura 1 muestra el flujo del programa: en morado la inicialización y en naranja lo que se repite en cada cuadro.

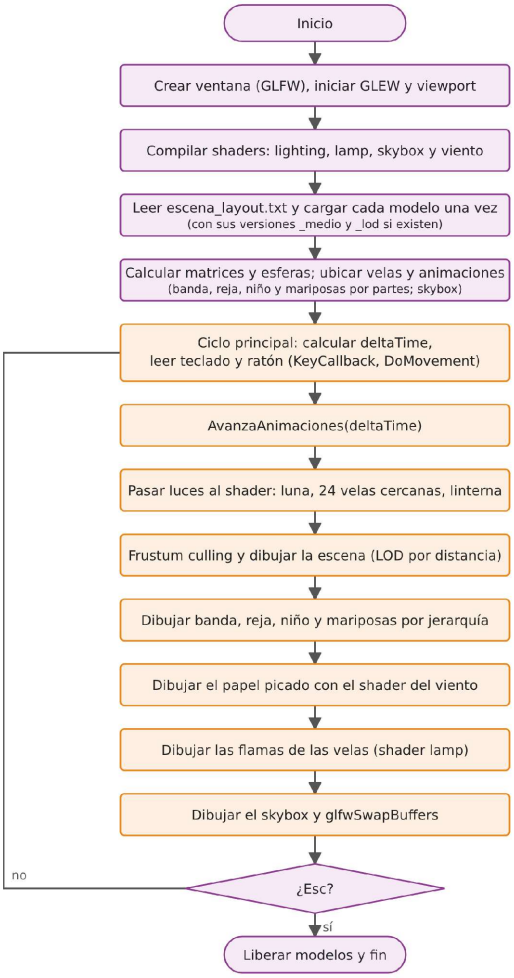


Figura 1: Diagrama de flujo del programa.

4.3 Estructura de archivos y carpetas

Ruta	**Contenido**
`ProyectoFinal.slnx / .vcxproj`	Solución de Visual Studio (Release | Win32)
`src/main.cpp`	Programa principal: escena, luces, animaciones e interacción
`include/`	Código base del laboratorio: `Shader.h`, `Camera.h`, `Mesh.h`, `Model.h`, SOIL2, `stb_image.h`
`shaders/lighting.vs/.frag`	Phong (luna, 24 velas, linterna), Fresnel, niebla, tinte y transparencia
`shaders/viento.vs`	Igual que `lighting.vs`, pero mueve el papel picado (animación 1)
`shaders/lamp.vs/.frag`	Flamas de las velas (color sólido)
`shaders/skybox.vs/.frag`	Cubo del cielo nocturno con niebla en el horizonte

Ruta	Contenido
`assets/escena_layout.txt`	Un renglón por objeto de la escena
`assets/animaciones_layout.txt`	Lugar de la banda y del niño que saluda
`assets/skybox/`	Seis caras del cielo (right, left, top, bottom, front, back)
`assets/models/<modelo>/`	`<modelo>.obj`, `.mtl` y texturas PNG (y `_medio.obj`, `_lod.obj` si tiene)
`assets/models/banda/`	Siete músicos por partes con su `pivotes.txt` y `banda_layout.txt`
`assets/models/persona_nino_partes/`	Niño por partes (animación 4)
`assets/models/reja/`	Las dos hojas de la reja con el origen en la bisagra (animación 3)
`assets/models/mariposa/`	Cuerpo y alas de la mariposa monarca (animación 5)
`external/`	GLEW, GLFW, assimp y GLM
`build/`	`ProyectoFinal.exe` con `glew32.dll`, `assimp-vc140-mt.dll`, `msvcp140.dll` y `vcruntime140.dll`
`docs/`	Manuales e imágenes de referencia

Tabla 2: Estructura del repositorio.

4.4 Metodología de software aplicada

Usamos una metodología de prototipos evolutivos con entregas incrementales. Primero hicimos un prototipo que solo cargaba los modelos, y en cada iteración le agregamos una parte nueva, la probamos y la corregimos antes de pasar a la siguiente. Cada avance se subió a GitHub el día en que se terminó, alternando entre los dos integrantes, para que el historial del repositorio documente el progreso real (tabla 4).

Fase	Qué se hizo
1. Requisitos y planeación	Elegimos el tema (panteón en Día de Muertos con una ofrenda), las imágenes de referencia y los objetos a recrear. Revisamos el PDF del proyecto y repartimos el trabajo por días.
2. Diseño	Definimos la distribución del panteón (entrada, camino, tumbas a los lados y ofrenda frente a la capilla), las luces y las cinco animaciones. Diseñamos el formato de `escena_layout.txt` para colocar los objetos.
3. Prototipo inicial	Estructura del repositorio, código base del laboratorio, shaders y skybox; después, los primeros modelos texturizados y una escena con luces.
4. Iteraciones	Agregamos por partes las terrazas, tumbas y velas; la ofrenda, coronas, tapetes y papel picado; las personas; y una por una las cinco animaciones. Después de cada parte compilamos, recorrimos la escena y corregimos objetos encimados, flotando o sin textura.

Fase	Qué se hizo
5. Pruebas	Revisamos cada tecla, cada animación y el rendimiento, y optimizamos el dibujo de la escena. Al final quitamos objetos para que la escena se viera menos saturada y reacomodamos tumbas y personas para que no quedaran huecos.
6. Entrega	Ejecutable en `build/`, manuales, video y exposición.

Tabla 3: Fases de la metodología.

4.5 Planificación y cronograma (diagrama de Gantt)

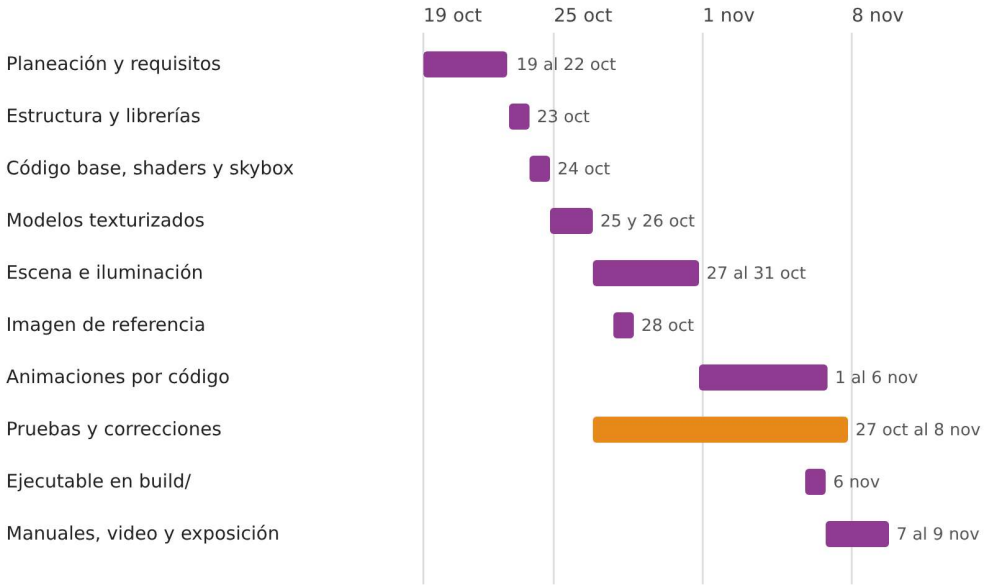


Figura 2: Diagrama de Gantt del proyecto. Las pruebas (en naranja) se hicieron a lo largo de todo el desarrollo.

Fecha	Responsable	Avance subido a GitHub
23 oct	Yasser	Estructura del proyecto y librerías
24 oct	Erick	Código base, shaders de iluminación y skybox
25 oct	Yasser	Escenario, capilla y primeros modelos
26 oct	Erick	Catrina, calaveritas, portarretratos y cempasúchil
27 oct	Yasser	Escena del panteón con luces y niebla
28 oct	Erick	Imagen de referencia
29 oct	Yasser	Terrazas, tumbas y velas
30 oct	Erick	Ofrenda, coronas, tapetes y papel picado
31 oct	Yasser	Personas

Fecha	Responsable	Avance subido a GitHub
1 nov	Erick	Animación 1: viento en el papel picado
2 nov	Yasser	Animación 2: banda
3 nov	Erick	Animación 3: reja
4 nov	Yasser	Animación 4: niño que saluda
6 nov	Erick	Animación 5: mariposas
6 nov	Yasser	Ejecutable en `build/`
7 nov	Erick	Manual técnico
8 nov	Yasser	Escena menos saturada, manual de usuario y README final

Tabla 4: Avances diarios subidos al repositorio.

5 Modelado y recursos gráficos

5.1 Descripción de los objetos modelados

Los objetos se modelaron y texturizaron en Blender y se exportaron a OBJ con sus texturas PNG. La tabla 5 lista los objetos de la imagen de referencia que recreamos.

Objeto	Carpeta	Descripción
Ofrenda	`ofrenda`	Ofrenda de varios niveles con manteles rosas, cajas laterales, banca y macetas.
Vela	`vela`, `flama`	Vela de cera con flama; cada una es una luz puntual.
Calaverita de azúcar	`calaverita`	Calavera con betún de colores.
Portarretratos	`portarretratos`	Marco de madera con foto.
Cempasúchil	`flor_cempasuchil`, `petalos_camino`	Flor con tallo y hojas, y caminos de pétalos.
Papel picado	`papel_picado`	Tiras de papel de colores con huecos (transparencia).
Pan de muerto	`pan_de_muerto`	Pan con huesitos y azúcar.
Platillos	`platillos_altar`	Platos, jarros, frutas y comida de la ofrenda.
Florero	`florero`	Florero de porcelana blanca con cempasúchil.
Catrina	`catrina`	Catrina con vestido negro floreado y sombrero con plumas.

Tabla 5: Objetos principales de la recreación.

Para ambientar el panteón agregamos el escenario con barda y reja, la capilla, las terrazas con escaleras, tumbas de tres tipos, mausoleos, cruces, un ángel, angelitos, árboles, copaleros, coronas de funeraria (seis tipos), tapetes de flores (siete diseños), guirnaldas, personas (diez variantes de hombre, mujer, niño y niña), la banda y las mariposas.

Además de la imagen de la ofrenda completa (figura 9), para cada uno de los diez objetos principales buscamos una foto real parecida a lo que íbamos a modelar y la usamos como guía de forma, proporción y color. En la figura 3 está cada foto junto a su modelo.

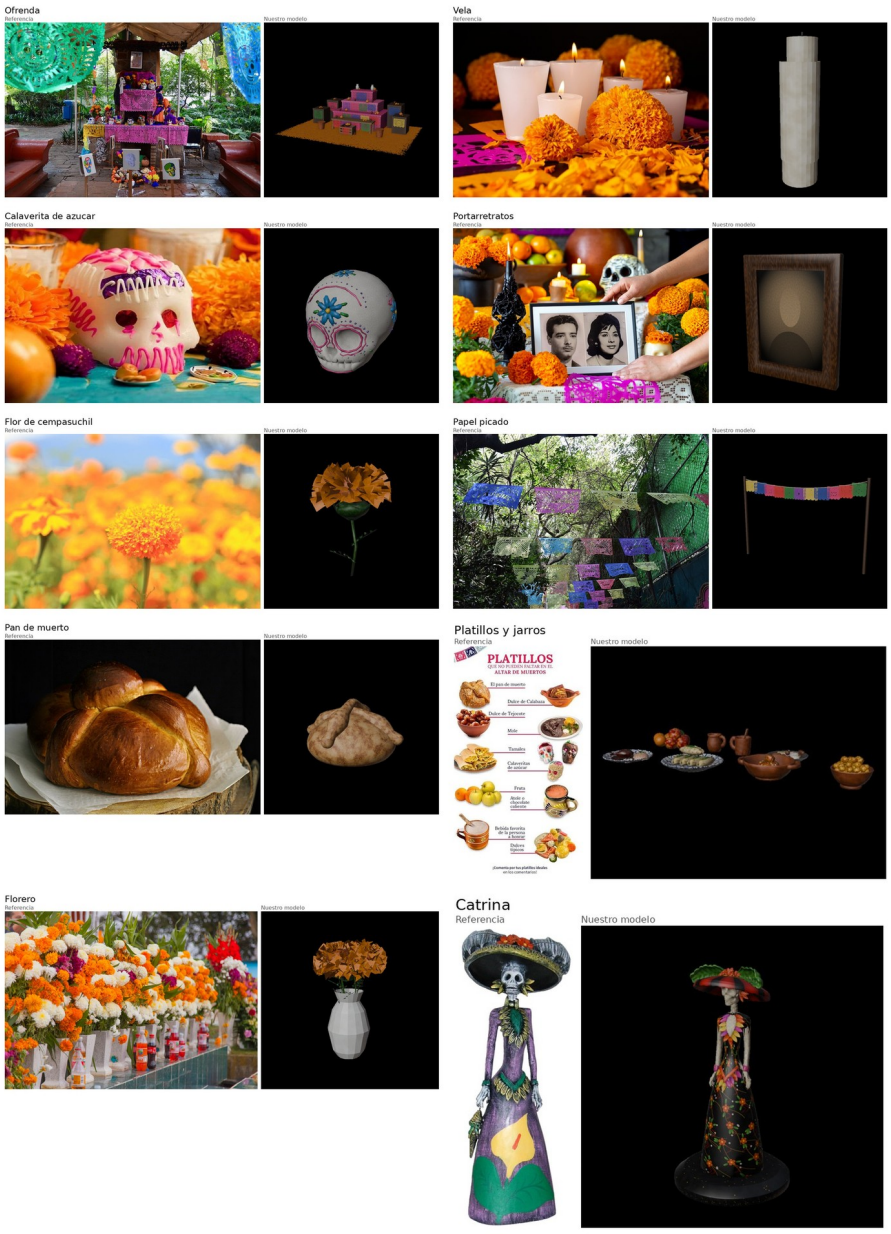


Figura 3: Foto de referencia (izquierda) y modelo terminado (derecha) de cada objeto.

Las fotos de páginas web tienen derechos reservados de sus autores; solo las usamos como referencia visual y citamos su fuente en la tabla 6 y en la sección 10. Las de Wikimedia Commons tienen licencia Creative Commons.

Objeto	Fuente de la foto	Autor o sitio	Licencia
Ofrenda	Ofrenda de Día de Muertos in Coyoacán	Inés Suárez R. (Wikimedia Commons)	CC BY-SA 4.0
Vela	Cuatro consejos para usar la flor de cempasúchil en la ofrenda	ADN40 (2024)	Derechos reservados
Calaverita de azúcar	Cómo hacer calaveritas de azúcar	TV Azteca Jalisco	Derechos reservados
Portarretratos	Aura Frames quiere reunir más recuerdos familiares en las ofrendas	TechRadar México	Derechos reservados
Cempasúchil	6 cosas que debes saber de la flor de cempasúchil	Vive USA	Derechos reservados
Papel picado	Papel picado del Día de Muertos	Carmalvi (Wikimedia Commons)	CC BY-SA 4.0
Pan de muerto	Receta de pan de muerto	Directo al Paladar	Derechos reservados
Platillos	Infografía “Platillos que no pueden faltar en el altar de muertos”	Autor no identificado (s.f.)	Derechos reservados
Florero	Tres flores indispensables para el altar de Día de Muertos	Meteored México	Derechos reservados
Catrina	Catrina sculpture	Dominik (Wikimedia Commons)	CC BY-SA 3.0

Tabla 6: Fuentes y licencias de las fotos de referencia.

Cada modelo distinto se carga una sola vez (de `assets/models/<modelo>/<modelo>.obj`) y se dibuja en cada posición con la matriz

$$\text{model} = T(x, y, z) \cdot R_y(\text{rotY}) \cdot S(\text{escala}).$$

Los datos salen de `assets/escena_layout.txt`, donde cada renglón coloca un objeto:

```
# modelo x y z rotY(grados) escala [tinte r g b]
ofrenda 0.000 0.000 -1.950 0.0 1.000
tumba -5.331 0.450 8.474 96.9 0.347 1.05 1.05 1.05
```

Las coordenadas son de OpenGL (Y hacia arriba), en metros. El tinte opcional multiplica el color de la textura y sirve para pintar tumbas y velas de distintos colores con el mismo modelo. Cada renglón de vela se vuelve además una luz puntual.

Los modelos animados por jerarquía (músicos y niño) están separados en un OBJ por parte, con el origen de cada parte en su articulación, igual que el perro de la práctica Máquina de estados. El archivo `pivotes.txt` dice dónde está cada articulación y quién es su padre; los padres van antes que los hijos:

```
# parte pivote_x pivote_y pivote_z padre
torso 0.0 0.6072 0.0 -
cabeza 0.0 1.0131 -0.0053 torso
brazoder -0.1155 0.9438 0.0 torso
antebrazoder -0.1327 0.7689 0.0026 brazoder
```

Por su parte, `assets/animaciones_layout.txt` da la posición y el giro de la banda y del niño que saluda, y `assets/models/banda/banda_layout.txt` acomoda a los siete músicos en arco respecto al centro de la banda.

5.2 Texturas y materiales

Las texturas son archivos PNG que se cargan con SOIL2 junto con cada OBJ y su `.mtl`. En este proyecto, `Model.h` las carga en RGBA en lugar de RGB, porque el papel picado y las alas de las mariposas usan transparencia (canal alfa): el shader descarta los píxeles con alfa menor a 0.5. El color de la textura se multiplica por el tinte del layout y, si se ve la cara de atrás de una flor o un pétalo, se voltea la normal para que se ilumine igual. El cielo usa seis imágenes (right, left, top, bottom, front, back) cargadas en un `GL_TEXTURE_CUBE_MAP`.

5.3 Iluminación y ambientación

`lighting.frag` es el shader de Fuentes de luz con el modelo de Phong: para cada luz suma la componente ambiental, la difusa y la especular.

$$I = I_a\, k_d + I_d\, k_d\ \text{máx}(N \cdot L, 0) + I_s\, k_s\ \text{máx}(R \cdot V, 0)^n$$

Luz	Tipo	Valores
Luna	Direccional	Dirección $(-0.4, -0.8, 0.45)$, color azulado $(0.45, 0.50, 0.75)$ por la intensidad de la luna (teclas + y −, de 0 a 2).
Velas	24 puntuales	Color cálido $(1, 0.55, 0.2)$ por el parpadeo; atenuación $1/(1 + 0.7d + 1.8d^2)$. Se mandan al shader las 24 velas más cercanas a la cámara (de 273).
Linterna	Spotlight en la cámara	Cono de 12° con borde suave hasta 18°; atenuación $1/(1 + 0.09d + 0.032d^2)$.

Tabla 7: Fuentes de luz de la escena.

Además del Phong del laboratorio, el shader agrega:

- **Fresnel.** Un brillo azulado de luna en los bordes de los objetos, que crece cuando la superficie se ve de canto. Se basa en la aproximación de Schlick (1994): $F = (1 - \text{máx}(N \cdot V, 0))^4$, con fuerza 0.35 por la intensidad de la luna.
- **Niebla.** Exponencial al cuadrado con la distancia (Microsoft, s.f.) y más espesa cerca del suelo:
$$f = \left(1 - e^{-(0.085\, d)^2}\right) \cdot \left(0.55 + 0.45\, e^{-y/3}\right).$$
El color final se mezcla con el de la niebla según f.

- **Caras dobles.** Si se ve la cara de atrás de una flor o un pétalo, se voltea la normal para que se ilumine igual.
- **Transparencia.** Los píxeles con alfa menor a 0.5 se descartan (huecos del papel picado y contorno de las alas).
- **Tinte por objeto.** El color de la textura se multiplica por el tinte del layout.

El skybox es un cubo con un `GL_TEXTURE_CUBE_MAP` de seis caras nocturnas. Se dibuja al final con `glDepthFunc(GL_LEQUAL)` y sin la traslación de la cámara, y en el horizonte se mezcla con el color de la niebla. Las flamas se dibujan con `lamp.frag` en color sólido, y su altura y color cambian con el mismo parpadeo de la luz de su vela.



Figura 4: Escena con Fresnel.



Figura 5: Escena sin Fresnel (tecla R).

6 Documentación del código fuente

6.1 Inicialización de OpenGL

Al iniciar, el programa crea la ventana con GLFW, inicia GLEW, configura el viewport y compila los cuatro shaders (lighting, lamp, skybox y viento). Después lee `escena_layout.txt`, carga cada modelo una sola vez (con sus versiones `_medio` y `_lod` si existen), calcula las matrices y esferas envolventes, ubica las velas y las animaciones (banda, reja, niño y mariposas por partes) y carga el skybox. El flujo completo está en la figura 1.

Si el programa no encuentra `assets/` en la carpeta actual (por ejemplo, al abrir el `.exe` con doble clic dentro de `build/` o desde otra carpeta), la función `BuscaCarpetaDelProyecto` toma la ruta del ejecutable (`argv[0]`) y busca `assets/escena_layout.txt` en la carpeta del `.exe`, en la de arriba y en la de dos niveles arriba. Con `chdir` cambia la carpeta de trabajo a la primera donde lo encuentra, y por eso el ejecutable funciona sin la carpeta Debug y sin Visual Studio.

`Shader.h` y `Camera.h` se usan sin cambios. A `Mesh.h` y `Model.h` les hicimos cambios pequeños para poder dibujar una escena de este tamaño (tabla 8).

Archivo	Cambio	Por qué
`Mesh.h`	Las ubicaciones de los uniforms de las texturas se buscan una sola vez por shader y se guardan.	El original llamaba a `glGetUniformLocation` en cada dibujo; con más de 1,200 objetos eso bajaba mucho los cuadros por segundo.
`Mesh.h`	Guarda la caja que envuelve cada malla (mínimo y máximo).	Para calcular la esfera envolvente de cada objeto (frustum culling).
`Mesh.h` y `Model.h`	`Draw(Shader &shader)` recibe el shader por referencia.	Evita copiar el objeto `Shader` en cada dibujo.
`Model.h`	Funciones `Centro()` y `Radio()`.	Esfera envolvente del modelo.
`Model.h`	Las texturas se cargan en RGBA en lugar de RGB.	El papel picado y las alas de las mariposas usan transparencia (canal alfa).

Tabla 8: Cambios al código base del laboratorio.

6.2 Sistema de renderizado

En cada cuadro, el ciclo principal calcula `deltaTime`, lee el teclado y el ratón, avanza las animaciones y pasa las luces al shader (luna, las 24 velas más cercanas y linterna). Después hace frustum culling y dibuja la escena (con nivel de detalle según la distancia), dibuja la banda, la reja, el niño y las mariposas por jerarquía, el papel picado con el shader del viento, las flamas de las velas con el shader lamp y, al final, el skybox, para terminar con `glfwSwapBuffers`. Para que esto corra con fluidez con más de 1,200 objetos aplicamos las optimizaciones de la tabla 9.

Técnica	Cómo funciona
Un modelo, muchas instancias	Cada modelo se carga una vez y se dibuja en todas sus posiciones. Los dibujos se ordenan por modelo para cambiar menos de texturas.
Matrices precalculadas	La matriz y la esfera envolvente de cada objeto se calculan una sola vez al iniciar.
Frustum culling	En cada cuadro se sacan los seis planos de la vista de projection · view (Gribb y Hartmann, 2001; Six, 2021); un objeto cuya esfera queda del lado de afuera de algún plano no se dibuja.
Niveles de detalle	A más de 4 m se usa la versión `_medio.obj` (personas, flores) y a más de 11 m la versión `_lod.obj`, con menos triángulos (Luebke et al., 2003).
Niebla	Con niebla, los objetos pequeños (radio menor a 1.5 m) a más de 40 m no se dibujan porque ya no se ven.
Luces	Solo las 24 velas más cercanas a la cámara se mandan al shader (con `std::partial_sort`; cppreference.com, s.f.).
Uniforms	`Mesh.h` guarda las ubicaciones de los uniforms en lugar de buscarlas en cada dibujo.

Tabla 9: Optimizaciones.

6.3 Cámara y perspectiva

La cámara es la de `Camera.h` del código base, sin cambios, y se mueve en primera persona (sección 6.4). La linterna es un spotlight que sale de la posición de la cámara y apunta hacia donde ve. El frustum culling usa los seis planos extraídos de la matriz projection · view de cada cuadro, y el skybox se dibuja sin la traslación de la cámara para que parezca estar infinitamente lejos.

6.4 Manejo de entradas del usuario

`DoMovement()` mueve la cámara con W, A, S, D o las flechas y cambia la luz de la luna con + y −. `MouseCallback()` gira la cámara con el ratón. `KeyCallback()` guarda en el arreglo `keys` qué teclas están presionadas (para caminar) y, al presionar una tecla, cambia la bandera correspondiente (tabla 10); también cierra el programa con Esc.

Tecla	Variable	Efecto
F	`linternaOn`	Spotlight con o sin intensidad
V	`veladorasOn`	Luces puntuales y flamas
R	`fresnelOn`	Fuerza del Fresnel en 0 o 0.35
N	`nieblaOn`	Niebla en el shader y distancia de dibujo
+ / −	`lunaIntensidad`	Sube o baja 0.5 por segundo (en `DoMovement`)
1 a 5	`vientoOn`, `bandaOn`, `rejaAbierta`, `saludoOn`, `mariposasOn`	Animaciones

Tabla 10: Teclas que cambian la iluminación y las animaciones.

6.5 Funciones principales

Estructura	Contenido
`Instancia`	Un renglón de `escena_layout.txt`: modelo, posición, giro, escala y tinte.
`Dibujo`	Lo que se dibuja de cada instancia: el modelo y sus versiones `_medio` y `_lod`, la matriz ya calculada, la esfera envolvente (centro y radio) y el tinte.
`Parte`	Una parte de un modelo jerárquico: nombre, modelo, pivote y padre.
`FRAME`	Un keyframe de una mariposa (posición x, y, z), como en `KeyFrames.cpp`.

Tabla 11: Estructuras de `main.cpp`.

Función	Qué hace
`CargaLayout(ruta)`	Lee `escena_layout.txt` y regresa la lista de instancias.
`MatrizModelo(i)`	Calcula $T \cdot R_y \cdot S$ de una instancia.
`Parpadeo(i, t)`	Intensidad de la flama de la vela i en el tiempo t: $0.80 + 0.12\sin(7.3t + 1.7i) + 0.08\sin(13.1t + 4.1i)$. Cada vela tiene su propia fase.
`CargaPartes(carpeta)`	Lee `pivotes.txt` y carga el OBJ de cada parte.
`DibujaPartes(...)`	Dibuja un modelo por partes: la matriz de cada parte es la de su padre por T(offset desde el padre) $\cdot$ R(ángulos de la parte).
`PoseMusico(...)`	Calcula los ángulos de cada parte de un músico en el tiempo t (animación 2).
`CatmullRom(p0, p1, p2, p3, u)`	Punto de la curva de Catmull-Rom entre `p1` y `p2` (animación 5).
`PosMariposa(m, u)`	Posición de la mariposa m en el tramo actual de sus keyframes.
`AvanzaAnimaciones(dt)`	Avanza el estado de las cinco animaciones una vez por cuadro.

Función	Qué hace
`DoMovement()`	Mueve la cámara con W, A, S, D o las flechas y cambia la luz de la luna con + y −.
`KeyCallback(...)`	Prende y apaga luces y animaciones (F, V, R, N, 1 a 5) y cierra con Esc.
`MouseCallback(...)`	Gira la cámara con el ratón.
`LoadCubemap(caras)`	Carga las seis imágenes del skybox en un `GL_TEXTURE_CUBE_MAP`.

Tabla 12: Funciones principales de `main.cpp`.

6.6 Generación del ejecutable

Para la entrega compilamos el proyecto en Release | Win32. Con esa configuración el `.exe` sale directo en `build/` (`OutDir`) y los archivos intermedios quedan en `obj/Release/`, que no se sube al repositorio. Un evento posterior a la compilación copia `glew32.dll` y `assimp-vc140-mt.dll` junto al `.exe`.

Para que el programa abriera en una computadora sin Visual Studio revisamos con `dumpbin /dependents` qué DLL pide el ejecutable, y aparecieron dos bibliotecas de depuración.

La primera era la DLL de assimp. La `assimp-vc140-mt.dll` que venía con el código base es una compilación Debug de assimp 4.1.0 y depende de `MSVCP140D.dll`, `VCRUNTIME140D.dll` y `ucrtbased.dll`, que solo existen donde está instalado Visual Studio y no se pueden redistribuir. Compilamos assimp 4.1.0 desde su código oficial en Release | Win32 con CMake, usando las opciones `BUILD_SHARED_LIBS=ON` y `LIBRARY_SUFFIX=-vc140-mt`, y reemplazamos la DLL y su `.lib` en `external/assimp/`. Los encabezados quedaron igual, porque el `config.h` generado es idéntico. La DLL nueva pesa 4.1 MB (la anterior pesaba 16 MB) y la escena ahora carga en unos 24 s; antes tardaba 90 s.

La segunda era `soil2-debug.lib`, que pide la biblioteca `MSVCRTD`. En Release la ignoramos con `IgnoreSpecificDefaultLibraries` y el ejecutable usa solo la CRT normal.

También copiamos a `build/` `msvcp140.dll` y `vcruntime140.dll` desde la carpeta de redistribuibles de Visual Studio (`VC\Redist\MSVC\...\x86\Microsoft.VC145.CRT`), así que no hace falta instalar el Visual C++ Redistributable. Las bibliotecas `api-ms-win-crt-*` ya vienen con Windows 10 y 11. El contenido final de `build/` está en la tabla 13.

Archivo	Tamaño	Depende de
`ProyectoFinal.exe`	287 KB	OpenGL32, glew32, assimp, MSVCP140, VCRUNTIME140 y DLL de Windows
`assimp-vc140-mt.dll`	4.1 MB	MSVCP140, VCRUNTIME140 y DLL de Windows
`glew32.dll`	381 KB	OpenGL32
`msvcp140.dll`	604 KB	VCRUNTIME140 y DLL de Windows
`vcruntime140.dll`	120 KB	DLL de Windows

Tabla 13: Archivos de `build/` y lo que necesita cada uno. Ninguna DLL es de depuración.

7 Sistema de animaciones

Todas las animaciones se calculan en el código con el tiempo de cada cuadro (`deltaTime`), así que van a la misma velocidad en cualquier computadora. Las sencillas siguen la idea de la práctica Animación básica (transformar un modelo según una variable que cambia con el tiempo); las complejas se basan en Máquina de estados y KeyFrames.

7.1 Animación sencilla 1: viento en el papel picado (tecla 1)

La variable `fuerzaViento` sube o baja 0.7 por segundo entre 0 y 1 según `vientoOn`, para que el viento llegue y se calme poco a poco. El papel picado se dibuja con `viento.vs`, que mueve cada vértice antes de transformarlo con una suma de senos, como se hace para simular olas en tiempo real (Finch, 2004):

$$\text{ola} = \sin(3.2t + 1.8x + \text{fase}) + 0.4\sin(7.1t + 4x + \text{fase})$$
$$z' = z + \text{viento} \cdot \text{ola} \cdot (0.5 \cdot \text{bajo} + 0.06) \cdot \text{centro}$$

`bajo` crece con la distancia al cordel y `centro` vale 1 a la mitad de la tira y 0 junto a los postes, así que el papel se mueve más abajo y en medio. Cada tira tiene una fase distinta para que no se muevan todas igual.

7.2 Animación sencilla 2: banda tocando (tecla 2)

Cada músico es un modelo jerárquico por partes (cuerpo, torso, brazos, cabeza e instrumento) dibujado con `DibujaPartes`. `PoseMusico` calcula sus ángulos con el tiempo a 120 golpes por minuto: el cuerpo se mece de lado a lado ($\pm 6°$) y rebota, el torso se inclina, la tarola y la tambora golpean con los brazos (hasta 35°), el trombón saca la vara y su brazo la sigue, y los instrumentos de viento suben un poco. `fuerzaBanda` va de 0 a 1 en medio segundo y multiplica todos los ángulos, para que empiecen y dejen de tocar sin cortes.

7.3 Animación sencilla 3: reja de la entrada (tecla 3)

Cada hoja de la reja es un OBJ con su origen en la bisagra. `tReja` avanza o regresa entre 0 y 3 segundos según `rejaAbierta`, y el ángulo se suaviza con la misma curva de `smoothstep` de GLSL (Khronos Group, s.f.) para que arranque y frene despacio:

$$u = \text{tReja}/3, \qquad \text{ángulo} = 95° \cdot u^2(3 - 2u)$$
$$\text{model}_{izq} = T(\text{bisagra}_{izq}) \cdot R_y(\text{ángulo}), \qquad \text{model}_{der} = T(\text{bisagra}_{der}) \cdot R_y(-\text{ángulo})$$



Figura 6: La reja abierta vista desde adentro del panteón.

7.4 Animación compleja 1: niño que saluda (tecla 4)

Se basa en la práctica Máquina de estados. El niño es un modelo por partes y su comportamiento depende del estado en que está (figura 7 y tabla 14).

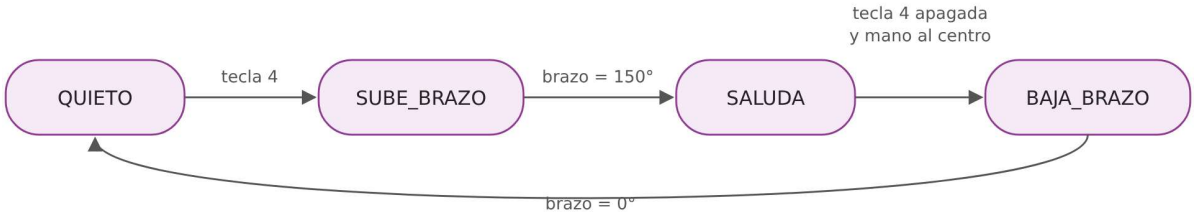


Figura 7: Máquina de estados del niño que saluda.

Estado	Qué hace
`QUIETO`	Brazo abajo. Si se activa la tecla 4 pasa a `SUBE_BRAZO`.
`SUBE_BRAZO`	El brazo derecho sube 150° por segundo y la cabeza se inclina hasta 8°. Al llegar a 150° pasa a `SALUDA`.
`SALUDA`	El antebrazo va de −25° a 25° y regresa (120° por segundo), cambiando de dirección con la bandera `manoDerecha`. Si se apaga la tecla y la mano está cerca del centro, pasa a `BAJA_BRAZO`.
`BAJA_BRAZO`	El brazo baja 120° por segundo y la cabeza regresa. Al llegar a 0° vuelve a `QUIETO`.

Tabla 14: Estados del niño que saluda.

La consideramos compleja porque combina estados, condiciones de cambio y la jerarquía de hombro, antebrazo y cabeza.

7.5 Animación compleja 2: mariposas monarca (tecla 5)

Se basa en la práctica KeyFrames. Cada una de las tres mariposas tiene un arreglo de tipo `FRAME` (`KeyFrame[m][MAX_FRAMES]`) con ocho posiciones alrededor de la ofrenda, y las variables `playIndex` (keyframe donde empieza el tramo) y `tramo` (de 0 a 1) avanzan con el tiempo: cada tramo dura 1 segundo y al terminar el último se regresa al primero.

En la práctica, la posición entre dos keyframes se interpola en línea recta. Aquí usamos una curva de Catmull-Rom (Boost C++ Libraries, s.f.; Catmull y Rom, 1974; Yuksel et al., 2011), que pasa por todos los keyframes y usa el anterior y el siguiente para que la trayectoria sea suave:

$$P(u) = \tfrac{1}{2}\Big[\,2P_1 + (P_2 - P_0)\,u + (2P_0 - 5P_1 + 4P_2 - P_3)\,u^2 + (3P_1 - P_0 - 3P_2 + P_3)\,u^3\Big]$$

Con la dirección de la curva se calcula hacia dónde voltea la mariposa (rumbo con `atan2`) y si va subiendo o bajando (cabeceo). La mariposa es un modelo jerárquico de tres partes: el cuerpo y dos alas hijas del cuerpo que giran sobre el eje Z entre 10° y 70° seis veces por segundo (ala $= 40° + 30°\sin(37.7t)$), y el cuerpo sube y baja un poco con cada aleteo. La consideramos compleja porque junta keyframes, una trayectoria curva, la orientación calculada con la tangente y la jerarquía de las alas.



Figura 8: Mariposa monarca volando junto a la ofrenda.

8 Pruebas y resultados

8.1 Pruebas funcionales

Prueba	Resultado
Compilar en Release	Compila sin errores.
Abrir el `.exe` con doble clic desde `build/`	Encuentra `assets/` y `shaders/` y carga la escena.
Lanzar el `.exe` desde otra carpeta de trabajo (`%TEMP%`)	Encuentra `assets/`; la ventana responde a los 24 s.
`dumpbin /dependents` del `.exe` y de cada DLL	Ninguna depende de DLL de depuración (`MSVCP140D`, `VCRUNTIME140D`, `ucrtbased`).
Cada tecla de luz (F, V, R, N, + y −)	Cambia la iluminación como se espera.
Cada avance del repositorio	Después de cada commit el proyecto compila y corre.

Tabla 15: Pruebas funcionales.

8.2 Validación de objetos y animaciones

Prueba	Resultado
Cada animación (1 a 5)	Se prende y se apaga con su tecla; arranca y termina sin saltos.
Varias animaciones al mismo tiempo	Funcionan juntas sin afectarse.
Recorrido completo de la escena	Sin objetos flotando, encimados ni sin textura.
Objetos de la ofrenda	Revisamos que ningún objeto flote ni se encime con otro en los niveles, las cajas laterales y la banca.

Tabla 16: Validación de objetos y animaciones.

8.3 Evidencias del resultado final

La figura 9 compara la imagen de referencia de la ofrenda con su recreación en OpenGL. Hay más evidencias en la figura 3 (objeto por objeto) y en las capturas de las animaciones de la sección anterior.



Figura 9: Imagen de referencia y recreación en OpenGL.

9 Conclusiones

Recreamos un panteón mexicano en Día de Muertos con una ofrenda tradicional parecida a la imagen de referencia, usando el código base del laboratorio. En un solo programa juntamos casi todos los temas del curso: carga de modelos con texturas, iluminación de Phong con varias fuentes de luz, cube mapping para el cielo, modelado jerárquico y los tres tipos de animación de las prácticas.

Lo más difícil fue que la escena se viera llena y realista sin que el programa se volviera lento. Leer la escena de un archivo de datos nos permitió colocar más de 1,200 objetos sin repetir código, y con el frustum culling, los niveles de detalle y mandar al shader solo las velas más cercanas se puede recorrer con fluidez. Al final también vimos que más objetos no siempre se ven mejor: quitamos árboles, estatuas, personas y papel picado, y reacomodamos las tumbas, y el panteón se lee con más claridad.

En las animaciones aprendimos que para hacer una animación compleja bastó con combinar bien las ideas de las prácticas: una máquina de estados con jerarquía para el niño, y keyframes con una curva suave, orientación calculada y aleteo para las mariposas. Subir avances diarios a GitHub nos ayudó a probar cada parte antes de seguir y dejó documentado todo el desarrollo.

10 Referencias (APA 7)

Proyecto y técnicas

Boost C++ Libraries. (s.f.). *Catmull-Rom splines*. En *Boost.Math*. Recuperado el 6 de noviembre de 2026, de `https://www.boost.org/doc/libs/latest/libs/math/doc/html/math_toolkit/catmull_rom.html`

Catmull, E., y Rom, R. (1974). A class of local interpolating splines. En R. E. Barnhill y R. F. Riesenfeld (Eds.), *Computer aided geometric design* (pp. 317–326). Academic Press. `https://doi.org/10.1016/B978-0-12-079050-0.50020-5`

cppreference.com. (s.f.). *std::partial_sort*. Recuperado el 6 de noviembre de 2026, de `https://en.cppreference.com/w/cpp/algorithm/partial_sort`

Espinoza Urzúa, E. (2026). *Proyecto final: Laboratorio de Computación Gráfica* [Documento del curso]. Facultad de Ingeniería, UNAM.

Finch, M. (2004). Effective water simulation from physical models. En R. Fernando (Ed.), *GPU*

Gems: Programming techniques, tips, and tricks for real-time graphics (cap. 1). Addison-Wesley. https://developer.nvidia.com/gpugems/gpugems/part-i-natural-effects/chapter-1-effective-water-simulation-physical-models

Gribb, G., y Hartmann, K. (2001). *Fast extraction of viewing frustum planes from the world-view-projection matrix* [Manuscrito no publicado].

Khronos Group. (s.f.). *smoothstep: Perform Hermite interpolation between two values*. OpenGL 4 Reference Pages. Recuperado el 6 de noviembre de 2026, de https://registry.khronos.org/OpenGL-Refpages/gl4/html/smoothstep.xhtml

Luebke, D., Reddy, M., Cohen, J. D., Varshney, A., Watson, B., y Huebner, R. (2003). *Level of detail for 3D graphics*. Morgan Kaufmann.

Microsoft. (s.f.). *Fog formulas (Direct3D 9)*. Microsoft Learn. Recuperado el 6 de noviembre de 2026, de https://learn.microsoft.com/en-us/windows/win32/direct3d9/fog-formulas

Schlick, C. (1994). An inexpensive BRDF model for physically-based rendering. *Computer Graphics Forum, 13*(3), 233–246. https://doi.org/10.1111/1467-8659.1330233

Six, J. (2021). *Frustum culling*. LearnOpenGL. https://learnopengl.com/Guest-Articles/2021/Scene/Frustum-Culling

Yuksel, C., Schaefer, S., y Keyser, J. (2011). Parameterization and applications of Catmull-Rom curves. *Computer-Aided Design, 43*(7), 747–755. https://doi.org/10.1016/j.cad.2010.08.008

Fotos de referencia de los objetos

ADN40. (2024, 15 de octubre). *Cuatro consejos para usar la flor de cempasúchil en la ofrenda del Día de Muertos*. https://www.adn40.mx/es-tendencia/2024-10-15/cuatro-consejos-para-usar-la-flor-cempasuchil-ofrenda-del-dia-de-muertos

Carmalvi. (s.f.). *Papel picado del Día de Muertos* [Fotografía]. Wikimedia Commons. CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Papel_picado_del_D%C3%ADa_de_Muertos.jpg

Directo al Paladar. (s.f.). *Receta de pan de muerto*. https://www.directoalpaladar.com/recetas-de-panes/receta-de-pan-de-muerto

Dominik. (s.f.). *Catrina sculpture* [Fotografía]. Wikimedia Commons. CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:Catrina-sculpture.jpg

Meteored México. (s.f.). *Tres hermosas flores indispensables para adornar el altar de Día de Muertos en tu hogar*. https://www.meteored.mx/noticias/actualidad/tres-hermosas-flores-indispensables-para-adornar-el-altar-de-dia-de-muertos-en-tu-hogar.html

Platillos que no pueden faltar en el altar de muertos [Infografía]. (s.f.). Imagen difundida en redes sociales; no se identificó la publicación original.

Suárez R., I. (s.f.). *Ofrenda de Día de Muertos in Coyoacán* [Fotografía]. Wikimedia Commons. CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Ofrenda_de_D%C3%ADa_de_Muertos_in_Coyoac%C3%A1n.jpg

TechRadar México. (s.f.). *Aura Frames quiere reunir más recuerdos familiares en las ofrendas de Día de Muertos.* https://global.techradar.com/es-mx/cameras/photography/aura-frames-quiere-reunir-mas-recuerdos-familiares-en-las-ofrendas-de-dia-de-muertos

TV Azteca Jalisco. (s.f.). *Cómo hacer calaveritas de azúcar.* https://www.aztecajalisco.com/arte-y-cultura/como-hacer-calaveritas-de-azucar/

Vive USA. (s.f.). *Día de Muertos: 6 cosas que debes saber de la flor de cempasúchil.* https://www.viveusa.mx/compras/dia-de-muertos-6-cosas-que-debes-saber-de-la-flor-de-cempasuchil/

11 Repositorio del proyecto

El código fuente, los modelos y el historial de cambios del desarrollo están en GitHub:

https://github.com/VvyasservV/Ofrenda-dia-de-muertos-laboratorio-de-CG

El video explicativo del proyecto está en YouTube:

https://youtu.be/TYJ01hJs5oU